#ORDERS EDA

#Data Ingestion

```
orders= spark.read.table("online.shop.orders")
```

#Converting spark dataframe to pandas

```
import pandas as pd
import numpy as np
import matplotlib as plt

od= orders.toPandas()

display(od)
```

OrderID	CustomerID	OrderDate	ProductID	Quantity	Discount	PaymentMet
500001	103695	2025-08-28	2003	4.0	0.1	Gateway
500002	107935	2024-05-31	2014	1.0	0.1	Wallet
500003	105141	2025-08-10	2002	1.0	0.2	CardToCard
500004	108780	2024-10-11	2012	1.0	0.1	Gateway
500005	101187	2024-02-19	2008	2.0	0.0	Gateway
500006	105545	2024-07-16	2011	4.0	0.0	Gateway
500007	106418	2025-10-15	2017	1.0	0.05	Wallet
500008	108755	2024-12-03	2014	1.0	0.0	Gateway
500009	105133	2025-04-15	2004	1.0	0.1	Gateway

EDA

```
#shows the number of rows and columns
od.shape
```

```
(50120, 8)
```

```
#extract the summary of stats from the numerical data
od.describe()
```

	OrderID	CustomerID	ProductID	Quantity	Discount
count	50120.000000	50120.000000	50120.000000	50040.000000	49899.000000
mean	525003.244872	105543.821788	2009.458799	1.881455	0.075130

	OrderID	CustomerID	ProductID	Quantity	Discount
std	14433.904231	22079.891005	5.577743	1.078846	0.077492
min	500001.000000	100001.000000	2001.000000	-2.000000	0.000000
25%	512503.750000	102515.000000	2004.000000	1.000000	0.000000
50%	525006.500000	105017.000000	2008.000000	2.000000	0.050000
75%	537501.250000	107510.000000	2014.000000	2.000000	0.100000
max	550000.000000	999999.000000	2020.000000	5.000000	0.300000

```
#summary of each column in the table
# Notice that OrderDate is a text

od.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 50120 entries, 0 to 50119
Data columns (total 8 columns):
 #   Column         Non-Null Count  Dtype
---  ------         --------------  -----
 0   OrderID        50120 non-null  int64
 1   CustomerID     50120 non-null  int64
 2   OrderDate      50085 non-null  object
 3   ProductID      50120 non-null  int64
 4   Quantity       50040 non-null  float64
 5   Discount       49899 non-null  float64
 6   PaymentMethod  49668 non-null  object
 7   Status         50120 non-null  object
dtypes: float64(2), int64(3), object(3)
memory usage: 3.1+ MB
```

```
#sum of duplicates in the table

od.duplicated().sum()
```

```
np.int64(120)
```

```
od.duplicated()    #checks for duplicates in each row. FALSE= no duplicate.
         TRUE= duplicated row
```

```
0        False
1        False
2        False
3        False
4        False
         ...
50115     True
50116     True
50117     True
50118     True
50119     True
Length: 50120, dtype: bool
```

```
#Counting the NULL values in each column
od.isnull().sum()
```

```
OrderID            0
CustomerID         0
OrderDate         35
```

```
ProductID          0
Quantity          80
Discount         221
PaymentMethod    452
Status             0
dtype: int64
```

```
#Fixing the null values in the table
od["Discount"] = od["Discount"].fillna(0)
```

```
#Fixing the null values in the table
od["PaymentMethod"] = od["PaymentMethod"].fillna("Unknown")
```

```
# Investigating the null values on the Quantity Column
od[od["Quantity"].isnull()]
```

	OrderID	CustomerID	OrderDate	ProductID	Quantity	Discount	Paym
1725	501726	103832	2024-03-17	2006	NaN	0.05	Gatev
2664	502665	106550	2025-06-24	2003	NaN	0.10	CardT
3521	503522	105703	2024-05-08	2005	NaN	0.00	Walle
4801	504802	104924	2024-08-18	2003	NaN	0.30	Walle
5312	505313	107357	2024-02-07	2019	NaN	0.05	Gatev
...	...	...	...	...	...	...	...
46972	546973	103620	2024-05-28	2014	NaN	0.10	CardT
48392	548393	109156	2025-05-08	2007	NaN	0.10	Walle
49467	549468	101765	2025-07-30	2007	NaN	0.00	Gatev
49566	549567	101615	2024-08-30	2014	NaN	0.05	Walle
49895	549896	108537	2026-01-31	2011	NaN	0.10	Gatev

80 rows × 8 columns

```
# Since the the status shows mostly "Completed" and not "Canceled", it does
        not make sense to replace NULLL values in the Quantity column with
        a zero.
# check the status distribution of the rows where the Quantity is NULL.
# Most orders are completed, then the NULl values under "Quantity" remains
        NULL
```

```
od[od["Quantity"].isnull()]["Status"].value_counts()
```

```
Status
Completed    72
Cancelled     7
Returned      1
Name: count, dtype: int64
```

```
#Investigating order date
# Check details/rows where the date shows NULL
```

```
od[od["OrderDate"].isnull()]
```

	OrderID	CustomerID	OrderDate	ProductID	Quantity	Discount	Paym
2355	502356	102438	None	2013	4.0	0.00	Gatev
4926	504927	102111	None	2004	1.0	0.00	Gatev
5564	505565	103942	None	2003	2.0	0.15	CardT
6267	506268	104260	None	2013	1.0	0.05	Cash
7647	507648	100075	None	2013	2.0	0.05	Walle
7859	507860	106056	None	2008	3.0	0.05	Gatev
9136	509137	102701	None	2013	2.0	0.15	Walle
10412	510413	109841	None	2006	4.0	0.00	Gatev
10559	510560	100699	None	2013	1.0	0.00	CardT
14165	514166	109973	None	2013	4.0	0.00	CardT
16434	516435	103600	None	2003	1.0	0.15	Gatev
17957	517958	103693	None	2017	4.0	0.05	Walle
18120	518121	108660	None	2013	4.0	0.00	Gatev
18599	518600	109568	None	2004	2.0	0.20	Walle
22527	522528	100139	None	2001	1.0	0.05	CardT
24983	524984	102500	None	2005	2.0	0.05	Gatev
25084	525085	106073	None	2006	1.0	0.05	Gatev
25706	525707	107146	None	2007	1.0	0.15	Gatev
26840	526841	104895	None	2010	1.0	0.00	Walle
27947	527948	104480	None	2004	4.0	0.00	Gatev
27980	527981	103491	None	2005	1.0	0.20	Walle
28316	528317	101891	None	2014	4.0	0.20	CardT
29081	529082	101067	None	2005	1.0	0.30	Gatev
30837	530838	104443	None	2009	2.0	0.00	Gatev
32695	532696	104803	None	2011	1.0	0.10	Gatev
33351	533352	108459	None	2004	1.0	0.00	Gatev
33796	533797	105799	None	2004	3.0	0.00	CardT
34249	534250	104846	None	2004	1.0	0.00	Gatev
34608	534609	100455	None	2008	2.0	0.00	Gatev
37795	537796	109466	None	2015	2.0	0.30	Cash
38862	538863	108570	None	2020	1.0	0.00	Gatev
42657	542658	106685	None	2013	3.0	0.00	Cash

	OrderID	CustomerID	OrderDate	ProductID	Quantity	Discount	Paym
46602	546603	101707	None	2016	3.0	0.05	Gatev
48266	548267	106816	None	2004	1.0	0.00	Gatev
49141	549142	109134	None	2010	1.0	0.10	Gatev

```
#Further investigation where the OrderDate is NULL
# I have decided to leave the NULL orderDate as NULL since the status is
        COMPLETED, this means all transactions were succesfully completed.
        It makes no sense to drop those NULL values because the outcome
        will be affected.
od[od["OrderDate"].isnull()][["OrderID", "Status", "Quantity",
        "PaymentMethod"]]
```

	OrderID	Status	Quantity	PaymentMethod
2355	502356	Completed	4.0	Gateway
4926	504927	Completed	1.0	Gateway
5564	505565	Completed	2.0	CardToCard
6267	506268	Completed	1.0	Cash
7647	507648	Completed	2.0	Wallet
7859	507860	Completed	3.0	Gateway
9136	509137	Completed	2.0	Wallet
10412	510413	Completed	4.0	Gateway
10559	510560	Completed	1.0	CardToCard
14165	514166	Completed	4.0	CardToCard
16434	516435	Completed	1.0	Gateway
17957	517958	Completed	4.0	Wallet
18120	518121	Completed	4.0	Gateway
18599	518600	Completed	2.0	Wallet
22527	522528	Completed	1.0	CardToCard
24983	524984	Completed	2.0	Gateway
25084	525085	Completed	1.0	Gateway
25706	525707	Completed	1.0	Gateway
26840	526841	Completed	1.0	Wallet
27947	527948	Returned	4.0	Gateway
27980	527981	Completed	1.0	Wallet
28316	528317	Completed	4.0	CardToCard
29081	529082	Completed	1.0	Gateway

	OrderID	Status	Quantity	PaymentMethod
30837	530838	Completed	2.0	Gateway
32695	532696	Completed	1.0	Gateway
33351	533352	Completed	1.0	Gateway
33796	533797	Completed	3.0	CardToCard
34249	534250	Completed	1.0	Gateway
34608	534609	Cancelled	2.0	Gateway
37795	537796	Completed	2.0	Cash
38862	538863	Completed	1.0	Gateway
42657	542658	Completed	3.0	Cash
46602	546603	Completed	3.0	Gateway
48266	548267	Completed	1.0	Gateway
49141	549142	Completed	1.0	Gateway

```
#Final check of null values
od.isnull().sum()
```

```
OrderID           0
CustomerID        0
OrderDate        35
ProductID         0
Quantity         80
Discount          0
PaymentMethod     0
Status            0
dtype: int64
```

```
od.count()          #counts the number of non null values in each column/
         Counting the total number of rows per column.
```

```
OrderID          50120
CustomerID       50120
OrderDate        50085
ProductID        50120
Quantity         50040
Discount         50120
PaymentMethod    50120
Status           50120
dtype: int64
```

```
# Where the booleon shows "True", it means there is duplicates. This code
         removes duplicated rows.
# Next step- check the shape of the table. Old= 50120   New= 50000 rows
od.drop_duplicates(inplace=True)

#Count of unique payment method types

od["PaymentMethod"].value_counts()
```

```
PaymentMethod
Gateway       23822
```

```
CardToCard    11946
Wallet         8782
Cash           5000
Unknown         450
Name: count, dtype: int64
```

```
#Counts the total of unique status type in the column
od["Status"].value_counts()
```

```
Status
Completed    45996
Cancelled     2472
Returned      1532
Name: count, dtype: int64
```

```
#Step 1: Convert the column to datetime format (just in case it is currently
         stored as text)
od['OrderDate'] = pd.to_datetime(od['OrderDate'])

# Step 2: Extract the componentsod['year'] = od['OrderDate'].dt.year
od['year'] = od['OrderDate'].dt.year
od['month'] = od['OrderDate'].dt.month
od['day'] = od['OrderDate'].dt.day
od['day_name'] = od['OrderDate'].dt.day_name()

import numpy as np

# Checks the text names directly
od['day_type'] = np.where(od['day_name'].isin(['Saturday', 'Sunday']),
         'Weekend', 'Weekday')

display(od)
```

OrderID	CustomerID	OrderDate	ProductID	Quantity	Discount	Payme
500001	103695	2025-08-28T00:00:00.000Z	2003	4.0	0.1	Gatew
500002	107935	2024-05-31T00:00:00.000Z	2014	1.0	0.1	Wallet
500003	105141	2025-08-10T00:00:00.000Z	2002	1.0	0.2	CardT
500004	108780	2024-10-11T00:00:00.000Z	2012	1.0	0.1	Gatew
500005	101187	2024-02-19T00:00:00.000Z	2008	2.0	0.0	Gatew
[illegible]	[illegible]	2024-07-	[illegible]	[illegible]	[illegible]	[illegible]

```
od.shape
```

```
(50000, 13)
```

I want to join the customers table to orders table on CustomerID. I think the first step is to call the customers to this notebook. Then inner join on CustomerID.

Orders + Customers = cts_od

1. Run the customer notebook to load the 'cts' dataframe into this session

```
%run "/Users/nomvelogcaba@gmail.com/Online_shop/Online_Shop_Case_Study/Data
        Processing/Customers EDA"
```

CustomerID	Age	City	SignupDate	CustomerSegment
100001	22.0	Tehran	2023-09-11	Regular
100002	55.0	Tabriz	2024-01-16	VIP
100003	49.0	Karaj	2025-07-31	New
100004	39.0	Tehran	2023-04-23	New
100005	38.0	Tehran	2023-04-29	Regular
100006	59.0	Isfahan	2023-11-27	Regular
100007	22.0	Karaj	2024-10-22	Regular
100008	51.0	Tabriz	2025-06-02	Regular
100009	27.0	Karaj	2024-09-08	Regular

```
(10000, 5)
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10000 entries, 0 to 9999
Data columns (total 5 columns):
 #   Column           Non-Null Count  Dtype  
---  ------           --------------  -----  
 0   CustomerID       10000 non-null  int64  
 1   Age              9820 non-null   float64
 2   City             9881 non-null   object 
 3   SignupDate       10000 non-null  object 
 4   CustomerSegment  10000 non-null  object 
dtypes: float64(1), int64(1), object(3)
memory usage: 390.8+ KB
```

```
<bound method NDFrame.describe of       CustomerID   Age     City  SignupDate
CustomerSegment
0         100001  22.0   Tehran  2023-09-11         Regular
1         100002  55.0   Tabriz  2024-01-16             VIP
2         100003  49.0    Karaj  2025-07-31             New
3         100004  39.0   Tehran  2023-04-23             New
4         100005  38.0   Tehran  2023-04-29         Regular
...          ...   ...      ...         ...             ...
9995      109996  40.0    Ahvaz  2025-07-16         Regular
9996      109997  65.0   Shiraz  2023-09-20         Regular
9997      109998  34.0   Tehran  2025-01-04         Regular
9998      109999  45.0  Mashhad  2023-02-13             VIP
9999      110000  47.0    Ahvaz  2023-06-04             New
```

```
[10000 rows x 5 columns]>

array(['Tehran', 'Tabriz', 'Karaj', 'Isfahan', 'Mashhad', 'Shiraz',
       'Rasht', 'Ahvaz', 'Kerman', 'Qom', 'tehran', None, 'Mashad'],
      dtype=object)

array(['Tehran', 'Tabriz', 'Karaj', 'Isfahan', 'Mashad', 'Shiraz',
       'Rasht', 'Ahvaz', 'Kerman', 'Qom', None], dtype=object)
```

	CustomerID	Age	City	SignupDate	CustomerSegment
9	100010	NaN	Mashad	2024-06-24	New
10	100011	NaN	Tehran	2025-08-11	Regular
16	100017	NaN	Isfahan	2024-12-08	Regular
153	100154	NaN	Tehran	2024-09-08	Regular
160	100161	NaN	Kerman	2025-05-23	Regular
...	...	...	...	...	...
9861	109862	NaN	Karaj	2025-12-26	Regular
9866	109867	NaN	Isfahan	2023-05-26	Regular
9882	109883	NaN	Tehran	2024-02-18	New
9928	109929	NaN	Isfahan	2025-09-17	Regular
9948	109949	NaN	Qom	2024-06-25	New

180 rows × 5 columns

```
min       18.000000
median    41.321487
max       65.000000
Name: Age, dtype: float64

array(['Regular', 'VIP', 'New'], dtype=object)

np.int64(0)
```

CustomerID	Age	City	SignupDate	CustomerSegment
100001	22.0	Tehran	2023-09-11T00:00:00.000Z	Regular
100002	55.0	Tabriz	2024-01-16T00:00:00.000Z	VIP
100003	49.0	Karaj	2025-07-31T00:00:00.000Z	New
100004	39.0	Tehran	2023-04-23T00:00:00.000Z	New
100005	38.0	Tehran	2023-04-29T00:00:00.000Z	Regular
[illegible]	[illegible]	[illegible]	2023-11-	[illegible]

```
# Now 'cts' is loaded from the other notebook, and you can merge it directly
cts_od = od.merge(cts, on="CustomerID", how="inner")

display(cts_od)
```

OrderID	CustomerID	OrderDate	ProductID	Quantity	Discount	Payme
500001	103695	2025-08-28T00:00:00.000Z	2003	4.0	0.1	Gatew
500002	107935	2024-05-31T00:00:00.000Z	2014	1.0	0.1	Wallet
500003	105141	2025-08-10T00:00:00.000Z	2002	1.0	0.2	CardT
500004	108780	2024-10-11T00:00:00.000Z	2012	1.0	0.1	Gatew
500005	101187	2024-02-19T00:00:00.000Z	2008	2.0	0.0	Gatew
[illegible]	[illegible]	2024-07-	[illegible]	[illegible]	[illegible]	[illegible]

```
cts_od.shape
```

```
(49970, 23)
```

#cts_od + products = cts_od_prd

Run the products notebook to load the 'prd' dataframe into this session

```
%run "/Users/nomvelogcaba@gmail.com/Online_shop/Online_Shop_Case_Study/Data
        Processing/Products EDA"
```

ProductID	ProductName	Category	UnitPrice
2001	Wireless Mouse	Electronics	18.0
2002	Mechanical Keyboard	Electronics	62.0
2003	USB-C Cable	Accessories	9.0
2004	Power Bank	Electronics	35.0
2005	Laptop Stand	Accessories	28.0
2006	Webcam	Electronics	48.0
2007	Headphones	Electronics	75.0
2008	Phone Case	Accessories	14.0
2009	Smart Watch	Wearables	120.0

```
array(['Wireless Mouse', 'Mechanical Keyboard', 'USB-C Cable',
       'Power Bank', 'Laptop Stand', 'Webcam', 'Headphones', 'Phone Case',
       'Smart Watch', 'Fitness Band', 'Desk Lamp', 'Office Chair',
       'Notebook', 'Backpack', 'Bluetooth Speaker', 'Monitor',
       'HDMI Cable', 'Microphone', 'Tablet', 'Gaming Controller'],
      dtype=object)
```

```
ProductName
Wireless Mouse         1
Mechanical Keyboard    1
Tablet                 1
Microphone             1
HDMI Cable             1
Monitor                1
Bluetooth Speaker      1
Backpack               1
Notebook               1
Office Chair           1
Desk Lamp              1
Fitness Band           1
Smart Watch            1
Phone Case             1
Headphones             1
Webcam                 1
Laptop Stand           1
Power Bank             1
USB-C Cable            1
Gaming Controller      1
Name: count, dtype: int64
```

```
Category
Electronics    9
Accessories    5
Wearables      2
Home Office    2
Stationery     1
Gaming         1
Name: count, dtype: int64
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 20 entries, 0 to 19
Data columns (total 4 columns):
 #   Column       Non-Null Count  Dtype  
---  ------       --------------  -----  
 0   ProductID    20 non-null     int64  
 1   ProductName  20 non-null     object 
 2   Category     20 non-null     object 
 3   UnitPrice    20 non-null     float64
dtypes: float64(1), int64(1), object(2)
memory usage: 772.0+ bytes
```

```
np.int64(0)
```

```
count     20.000000
mean      61.400000
std       62.836798
min        7.000000
25%       20.250000
50%       45.000000
75%       69.750000
max      260.000000
Name: UnitPrice, dtype: float64
```

```
# Now 'prd' is loaded from the other notebook, and you can merge it directly
cts_od_prd = cts_od.merge(prd, on="ProductID", how="inner")
```

```
display(cts_od_prd)
```

OrderID	CustomerID	OrderDate	ProductID	Quantity	Discount	Payme
500001	103695	2025-08-28T00:00:00.000Z	2003	4.0	0.1	Gatew
500002	107935	2024-05-31T00:00:00.000Z	2014	1.0	0.1	Wallet
500003	105141	2025-08-10T00:00:00.000Z	2002	1.0	0.2	CardT
500004	108780	2024-10-11T00:00:00.000Z	2012	1.0	0.1	Gatew
500005	101187	2024-02-19T00:00:00.000Z	2008	2.0	0.0	Gatew
[illegible]	[illegible]	2024-07-	[illegible]	[illegible]	[illegible]	[illegible]

```
cts_od_prd.shape
```

```
(49970, 26)
```

cts_od_prd + pmt = final_merged

#Run the payments notebook to load the 'pmt' dataframe into this session

```
%run "/Users/nomvelogcaba@gmail.com/Online_shop/Online_Shop_Case_Study/Data
        Processing/Payments EDA"
```

PaymentID	OrderID	PaymentDate	PaymentStatus
800001	500001	2025-08-28	Paid
800002	500002	2024-05-31	Paid
800003	500003	2025-08-10	Paid
800004	500004	2024-10-11	Paid
800005	500005	2024-02-19	Paid
800006	500006	2024-07-16	Failed
800007	500007	2025-10-15	Paid
800008	500008	2024-12-03	Paid
800009	500009	2025-04-15	Paid

```
(50000, 4)
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 50000 entries, 0 to 49999
Data columns (total 4 columns):
```

```
 #   Column         Non-Null Count  Dtype
---  ------         --------------  -----
 0   PaymentID      50000 non-null  int64
 1   OrderID        50000 non-null  int64
 2   PaymentDate    49965 non-null  object
 3   PaymentStatus  50000 non-null  object
dtypes: int64(2), object(2)
memory usage: 1.5+ MB
```

```
np.int64(0)
```

```
PaymentID         0
OrderID           0
PaymentDate      35
PaymentStatus     0
dtype: int64
```

```
PaymentStatus
Paid        46569
Failed       1924
Refunded     1507
Name: count, dtype: int64
```

```
# Now 'prd' is loaded from the other notebook, and you can merge it directly
final_merged = cts_od_prd.merge(pmt, on="OrderID", how="inner")

display(final_merged)
```

OrderID	CustomerID	OrderDate	ProductID	Quantity	Discount	Payme
500001	103695	2025-08-28T00:00:00.000Z	2003	4.0	0.1	Gatew
500002	107935	2024-05-31T00:00:00.000Z	2014	1.0	0.1	Wallet
500003	105141	2025-08-10T00:00:00.000Z	2002	1.0	0.2	CardT
500004	108780	2024-10-11T00:00:00.000Z	2012	1.0	0.1	Gatew
500005	101187	2024-02-19T00:00:00.000Z	2008	2.0	0.0	Gatew
[illegible]	[illegible]	2024-07-	[illegible]	[illegible]	[illegible]	[illegible]

Final_merged EDA

```
final_merged.shape
```

```
(49970, 29)
```

```
import numpy as np

# Calculate revenue for each order
final_merged['Revenue'] = final_merged['Quantity'] *
         final_merged['UnitPrice'] * (1 - final_merged['Discount'])
```

```
display(final_merged)
```

OrderID	CustomerID	OrderDate	ProductID	Quantity	Discount	Payme
500001	103695	2025-08-28T00:00:00.000Z	2003	4.0	0.1	Gatew
500002	107935	2024-05-31T00:00:00.000Z	2014	1.0	0.1	Wallet
500003	105141	2025-08-10T00:00:00.000Z	2002	1.0	0.2	CardT
500004	108780	2024-10-11T00:00:00.000Z	2012	1.0	0.1	Gatew
500005	101187	2024-02-19T00:00:00.000Z	2008	2.0	0.0	Gatew
[illegible]	[illegible]	2024-07-	[illegible]	[illegible]	[illegible]	[illegible]

#Final_merged EDA

```
import pandas as pd
import matplotlib.pyplot as plt

df= final_merged

#Check the table

df.head()
```

	OrderID	CustomerID	OrderDate	ProductID	Quantity	Discount	PaymentM
0	500001	103695	2025-08-28	2003	4.0	0.1	Gateway
1	500002	107935	2024-05-31	2014	1.0	0.1	Wallet
2	500003	105141	2025-08-10	2002	1.0	0.2	CardToCar
3	500004	108780	2024-10-11	2012	1.0	0.1	Gateway
4	500005	101187	2024-02-19	2008	2.0	0.0	Gateway

```
# Checking the rows ad columns of the combined table
df.shape

(49970, 30)

print(df["Status"].value_counts())

Status
Completed    45968
Cancelled     2471
```

```
Returned      1531
Name: count, dtype: int64
```

```
print(df["PaymentStatus"].value_counts())
```

```
PaymentStatus
Paid        46543
Failed       1922
Refunded     1505
Name: count, dtype: int64
```

Revenue calculation

We only recognise revenue when:

Status = Completed AND PaymentStatus = Paid

```
analysis_df = df[
    (df["Status"] == "Completed") &
    (df["PaymentStatus"] == "Paid") &
    (df["Quantity"].notna()) &
    (df["OrderDate"].notna())
].copy()

#Peak and least month revenue calculation
import pandas as pd

# 1. Your existing filtered dataframe cleanup
analysis_df = df[
    (df["Status"] == "Completed") &
    (df["PaymentStatus"] == "Paid") &
    (df["Quantity"].notna()) &
    (df["OrderDate"].notna())
].copy()

# 2. Ensure OrderDate is in datetime format
analysis_df["OrderDate"] = pd.to_datetime(analysis_df["OrderDate"])

# 3. Calculate Revenue per row (Adjust 'UnitPrice' if your column name is
        different)
# If you already have a 'TotalAmount' or 'Revenue' column, you can skip this
        step.
analysis_df["Revenue"] = analysis_df["Quantity"] * analysis_df["UnitPrice"]

# 4. Extract the Month-Year Period (e.g., '2026-01')
analysis_df["Month_Period"] = analysis_df["OrderDate"].dt.to_period("M")

# 5. Group by Month and sum the Revenue
monthly_revenue = analysis_df.groupby("Month_Period")["Revenue"].sum()

# 6. Find the peak and least months using idxmax() and idxmin()
peak_month = monthly_revenue.idxmax()
peak_revenue = monthly_revenue.max()

least_month = monthly_revenue.idxmin()
least_revenue = monthly_revenue.min()
```

```
# 7. Print the results
print(f"Peak Month: {peak_month} with Revenue of {peak_revenue:,.2f}")
print(f"Least Month: {least_month} with Revenue of {least_revenue:,.2f}")
```

```
Peak Month: 2024-03 with Revenue of 123,789.00
Least Month: 2026-02 with Revenue of 69,315.00
```

```
import pandas as pd

# 1. Your filtered dataframe cleanup
analysis_df = df[
    (df["Status"] == "Completed") &
    (df["PaymentStatus"] == "Paid") &
    (df["Quantity"].notna()) &
    (df["OrderDate"].notna())
].copy()

# 2. Ensure OrderDate is in datetime format
analysis_df["OrderDate"] = pd.to_datetime(analysis_df["OrderDate"])

# 3. Calculate Revenue per row (Change 'UnitPrice' if your column is named
        differently)
analysis_df["Revenue"] = analysis_df["Quantity"] * analysis_df["UnitPrice"]

# 4. Extract the Year
analysis_df["Year"] = analysis_df["OrderDate"].dt.year

# 5. Calculate Annual Revenue
# We convert the resulting Series to a DataFrame to make adding columns easy
annual_revenue = analysis_df.groupby("Year")["Revenue"].sum().to_frame()

# 6. Calculate the Year-over-Year (YoY) Percentage Difference
annual_revenue["YoY_Change_Percentage"] =
        annual_revenue["Revenue"].pct_change() * 100

# 7. Format for clean display
# Renames column for clarity and fills the first year's NaN value with 0 or a
        placeholder
annual_revenue["YoY_Change_Percentage"] =
        annual_revenue["YoY_Change_Percentage"].fillna(0)

print(annual_revenue)
```

```
        Revenue  YoY_Change_Percentage
Year
2024  1367829.0               0.000000
2025  1362087.0              -0.419789
2026   503415.0             -63.040907
```

```
print("Rows:", len(analysis_df))
print("Unique Orders:", analysis_df["OrderID"].nunique())
```

```
Rows: 42720
Unique Orders: 42720
```

#KPI Aggregation

```
total_revenue = analysis_df["Revenue"].sum()
```

```
total_orders = analysis_df["OrderID"].nunique()

total_units = analysis_df["Quantity"].sum()

aov = total_revenue / total_orders

print("Total Revenue:", round(total_revenue, 2))
print("Total Orders:", total_orders)
print("Total Units:", total_units)
print("AOV:", round(aov, 2))
```

```
Total Revenue: 3233331.0
Total Orders: 42720
Total Units: 80374.0
AOV: 75.69
```

```
print(df.shape)
print(df["OrderID"].nunique())

print(analysis_df.shape)
print(analysis_df["OrderID"].nunique())
```

```
(49970, 30)
49970
(42720, 31)
42720
```

```
#Checking order distribution with dfferent payment status and orders_status
print(
    df.groupby(["Status", "PaymentStatus"])
      .size()
      .reset_index(name="Orders")
)
```

```
      Status PaymentStatus  Orders
0  Cancelled        Failed      93
1  Cancelled          Paid    2307
2  Cancelled      Refunded      71
3  Completed        Failed    1751
4  Completed          Paid   42824
5  Completed      Refunded    1393
6   Returned        Failed      78
7   Returned          Paid    1412
8   Returned      Refunded      41
```

```
#Checking if OrderID is duplicated
print(
    df["OrderID"].duplicated().sum()
)
```

```
0
```

```
analysis_df["Year"] = analysis_df["OrderDate"].dt.year

analysis_df["Month"] = analysis_df["OrderDate"].dt.month

analysis_df["Month_Name"] = analysis_df["OrderDate"].dt.month_name()

analysis_df["Day_Name"] = analysis_df["OrderDate"].dt.day_name()
```

```
analysis_df["Year_Month"] = (
    analysis_df["OrderDate"]
    .dt.to_period("M")
    .astype(str)
)

display(analysis_df)
```

OrderID	CustomerID	OrderDate	ProductID	Quantity	Discount	Payme
500001	103695	2025-08-28T00:00:00.000Z	2003	4.0	0.1	Gatew
500002	107935	2024-05-31T00:00:00.000Z	2014	1.0	0.1	Wallet
500003	105141	2025-08-10T00:00:00.000Z	2002	1.0	0.2	CardT
500004	108780	2024-10-11T00:00:00.000Z	2012	1.0	0.1	Gatew
500005	101187	2024-02-19T00:00:00.000Z	2008	2.0	0.0	Gatew
[illegible]	[illegible]	2025-10-	[illegible]	[illegible]	[illegible]	[illegible]

```
#Monthly revenue check

monthly_revenue = (
    analysis_df
    .groupby("Year_Month")
    .agg(
        Revenue=("Revenue", "sum"),
        Orders=("OrderID", "nunique"),
        Units=("Quantity", "sum")
    )
    .reset_index()
)

monthly_revenue.head()
```

	Year_Month	Revenue	Orders	Units
0	2024-01	113789.0	1509	2852.0
1	2024-02	105062.0	1364	2553.0
2	2024-03	123789.0	1587	3014.0
3	2024-04	111433.0	1467	2765.0
4	2024-05	116351.0	1500	2841.0

#Monthly revenue

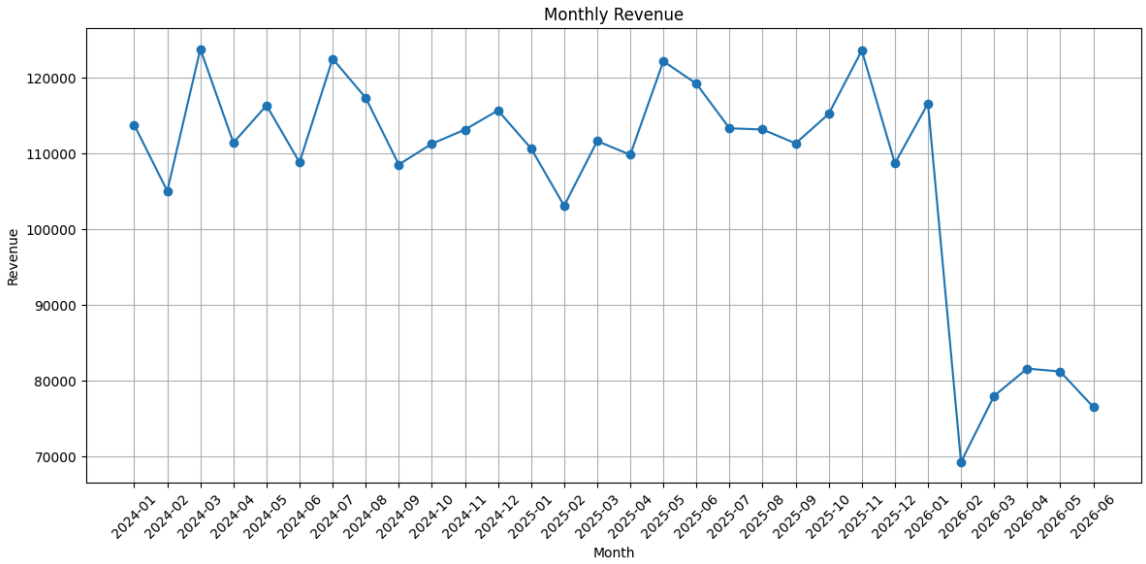
```
#Line graph for Monthly_Revenue

plt.figure(figsize=(12, 6))
```

```
plt.plot(
    monthly_revenue["Year_Month"],
    monthly_revenue["Revenue"],
    marker="o"
)

plt.title("Monthly Revenue")
plt.xlabel("Month")
plt.ylabel("Revenue")
plt.xticks(rotation=45)
plt.grid(True)

plt.tight_layout()
plt.show()
```



#Category revenue

```
#Category revenue

category_analysis = (
    analysis_df
    .groupby("Category")
    .agg(
        Revenue=("Revenue", "sum"),
        Units=("Quantity", "sum"),
        Orders=("OrderID", "nunique")
    )
    .reset_index()
)

#sort

category_analysis = category_analysis.sort_values(
    "Revenue",
    ascending=False
)

print(category_analysis)
```

```
      Category    Revenue    Units  Orders
1  Electronics  1635853.0  28632.0   15441
0  Accessories   521441.0  29991.0   15769
3  Home Office   421552.0   5163.0    2727
5    Wearables   404970.0   4977.0    2666
2       Gaming   187544.0   2758.0    1455
4   Stationery    61971.0   8853.0    4662
```

```
#Graph for Category Revenue

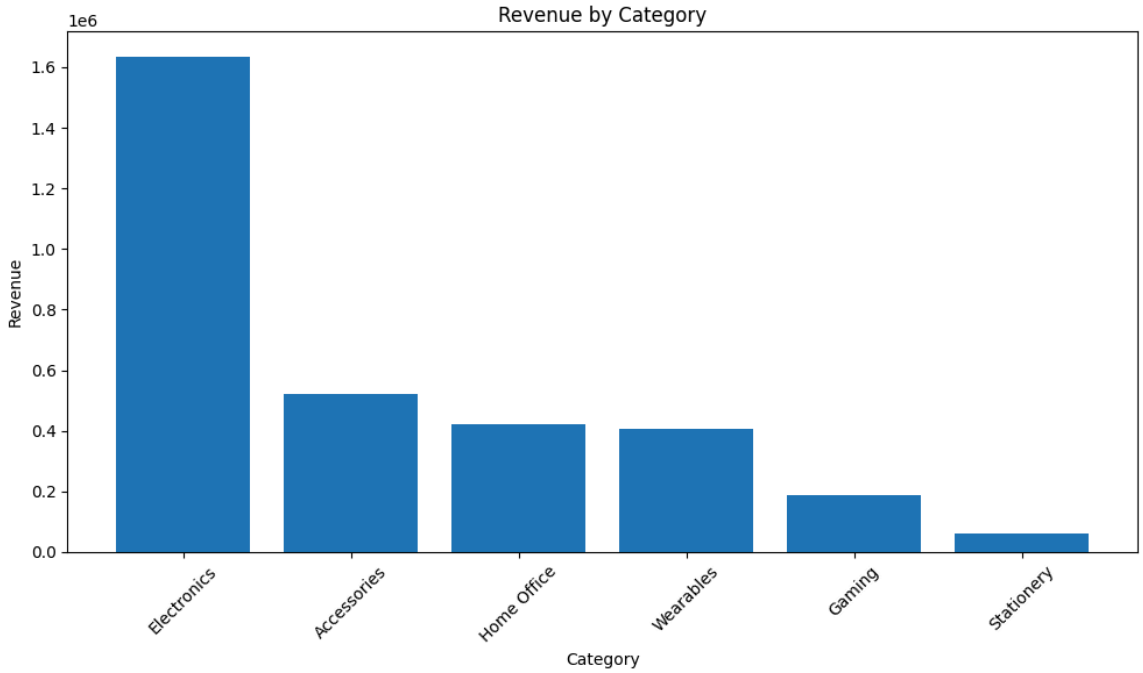
plt.figure(figsize=(10, 6))

plt.bar(
    category_analysis["Category"],
    category_analysis["Revenue"]
)

plt.title("Revenue by Category")
plt.xlabel("Category")
plt.ylabel("Revenue")

plt.xticks(rotation=45)

plt.tight_layout()
plt.show()
```



#Product Revenue

```
product_analysis = (
    analysis_df
    .groupby("ProductName")
    .agg(
        Revenue=("Revenue", "sum"),
        Units=("Quantity", "sum"),
        Orders=("OrderID", "nunique")
    )
    .reset_index()
```

```
)

#sort by descending revenue
product_revenue = product_analysis.sort_values(
    "Revenue",
    ascending=False
)

#print the top 10
top_10_products = product_revenue.head(10)
print(top_10_products)
```

```
            ProductName   Revenue   Units  Orders
6            Headphones  315000.0  4200.0    2245
12         Office Chair  311760.0  1732.0     906
16               Tablet  267020.0  1027.0     552
15          Smart Watch  242280.0  2019.0    1071
9            Microphone  219640.0  2584.0    1361
8   Mechanical Keyboard  210118.0  3389.0    1843
1     Bluetooth Speaker  198998.0  3431.0    1835
4     Gaming Controller  187544.0  2758.0    1455
14           Power Bank  167650.0  4790.0    2569
3          Fitness Band  162690.0  2958.0    1595
```

```
#Graph

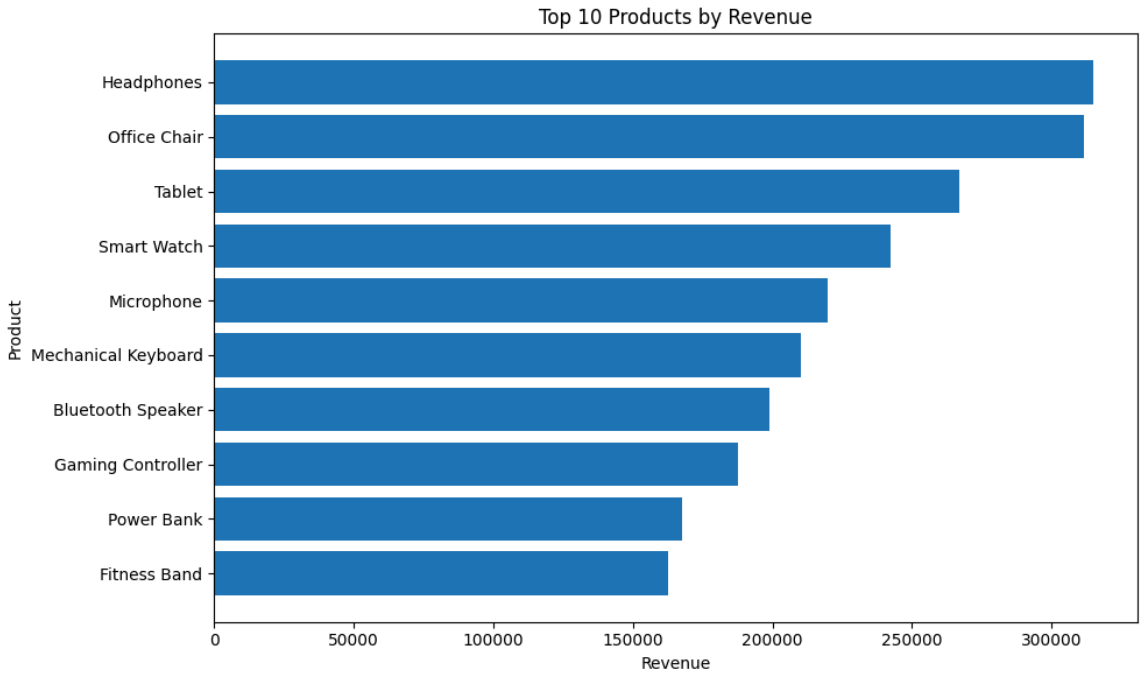
plt.figure(figsize=(10, 6))

plt.barh(
    top_10_products["ProductName"],
    top_10_products["Revenue"]
)

plt.title("Top 10 Products by Revenue")
plt.xlabel("Revenue")
plt.ylabel("Product")

plt.gca().invert_yaxis()

plt.tight_layout()
plt.show()
```



#Product Units Sold

```
product_units = product_analysis.sort_values(
    "Units",
    ascending=False
)

top_10_units = product_units.head(10)

print(top_10_units.to_string())
```

```
          ProductName   Revenue   Units  Orders
11           Notebook   61971.0  8853.0    4662
13         Phone Case  115318.0  8237.0    4300
17        USB-C Cable   69642.0  7738.0    4071
5          HDMI Cable   71027.0  6457.0    3382
19     Wireless Mouse   94410.0  5245.0    2779
14         Power Bank  167650.0  4790.0    2569
6          Headphones  315000.0  4200.0    2245
0            Backpack  161406.0  3843.0    2018
7        Laptop Stand  104048.0  3716.0    1998
1   Bluetooth Speaker  198998.0  3431.0    1835
```

```
#Graph

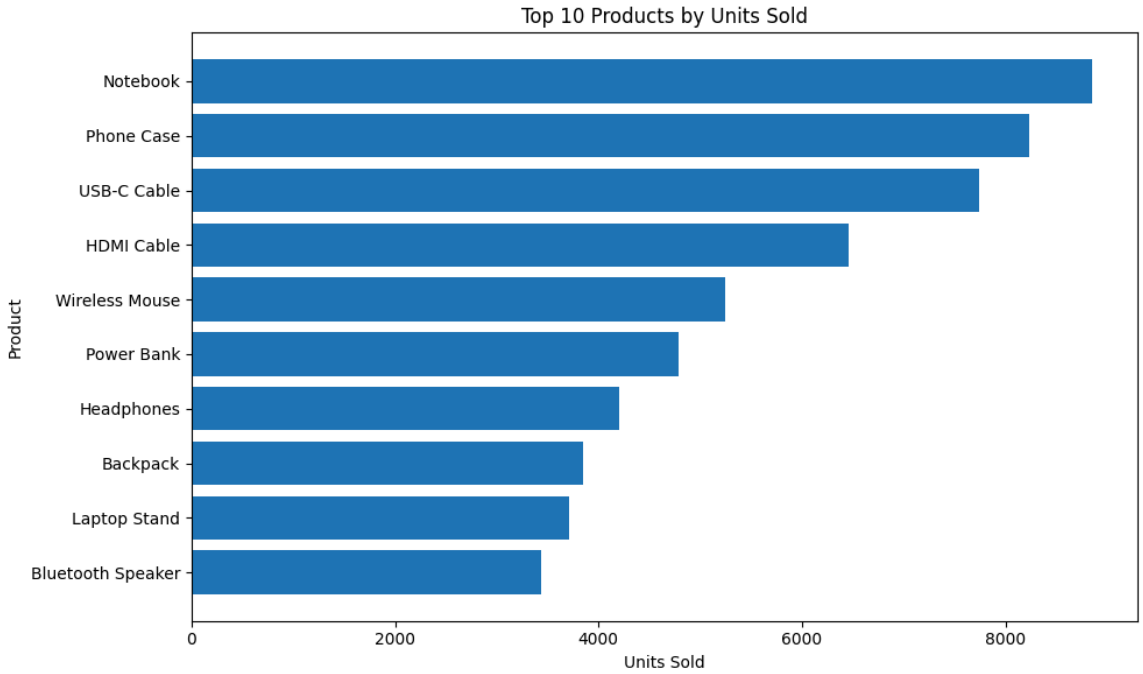
plt.figure(figsize=(10, 6))

plt.barh(
    top_10_units["ProductName"],
    top_10_units["Units"]
)

plt.title("Top 10 Products by Units Sold")
plt.xlabel("Units Sold")
plt.ylabel("Product")

plt.gca().invert_yaxis()
```

```
plt.tight_layout()
plt.show()
```



#City Revenue

```
city_analysis = (
    analysis_df
    .assign(
        City=analysis_df["City"].fillna("Uncategorized")
    )
    .groupby("City")
    .agg(
        Revenue=("Revenue", "sum"),
        Orders=("OrderID", "nunique"),
        Units=("Quantity", "sum")
    )
    .reset_index()
)
#sort
city_analysis = city_analysis.sort_values(
    "Revenue",
    ascending=False
)
top_10_cities = city_analysis.head(10)

print(top_10_cities)
```

```
      City   Revenue  Orders    Units
9    Tehran  891598.0   11960  22289.0
4   Mashad  406402.0    5331  10053.0
2    Karaj  322020.0    4245   7977.0
1  Isfahan  310737.0    4246   7924.0
7   Shiraz  292972.0    3760   7165.0
8   Tabriz  274322.0    3535   6702.0
0    Ahvaz  192429.0    2487   4715.0
6    Rasht  185734.0    2445   4627.0
```

```
3   Kerman  160552.0    2115   4053.0
5      Qom  153069.0    2054   3864.0
```

```
#Graph

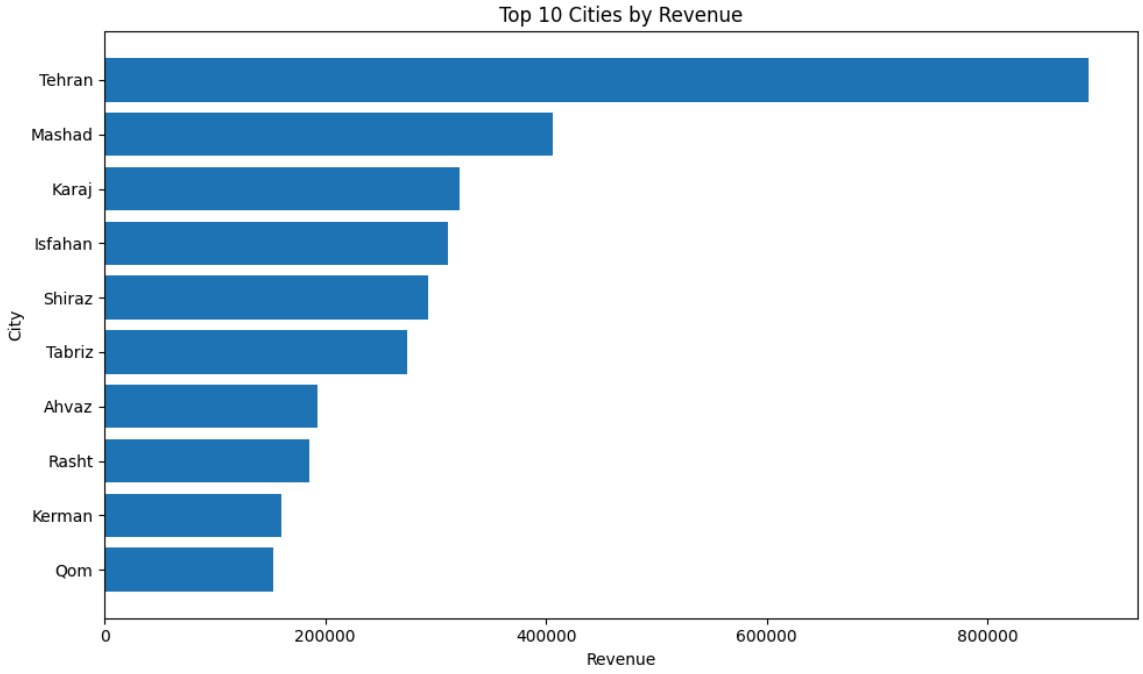
plt.figure(figsize=(10, 6))

plt.barh(
    top_10_cities["City"],
    top_10_cities["Revenue"]
)

plt.title("Top 10 Cities by Revenue")
plt.xlabel("Revenue")
plt.ylabel("City")

plt.gca().invert_yaxis()

plt.tight_layout()
plt.show()
```



#Customer Segment Revenue

```
segment_analysis = (
    analysis_df
    .groupby("CustomerSegment")
    .agg(
        Revenue=("Revenue", "sum"),
        Orders=("OrderID", "nunique"),
        Units=("Quantity", "sum")
    )
    .reset_index()
)

#Calculate Average Order Value (AOV)
segment_analysis["AOV"] = (
    segment_analysis["Revenue"] /
```

```
    segment_analysis["Orders"]
)

#Sort
segment_analysis = segment_analysis.sort_values(
    "Revenue",
    ascending=False
)

print(segment_analysis)
```

```
  CustomerSegment    Revenue  Orders    Units        AOV
1         Regular  1797102.0   23661  44462.0  75.952073
0             New  1123885.0   14824  28016.0  75.815232
2             VIP   312344.0    4235   7896.0  73.753011
```

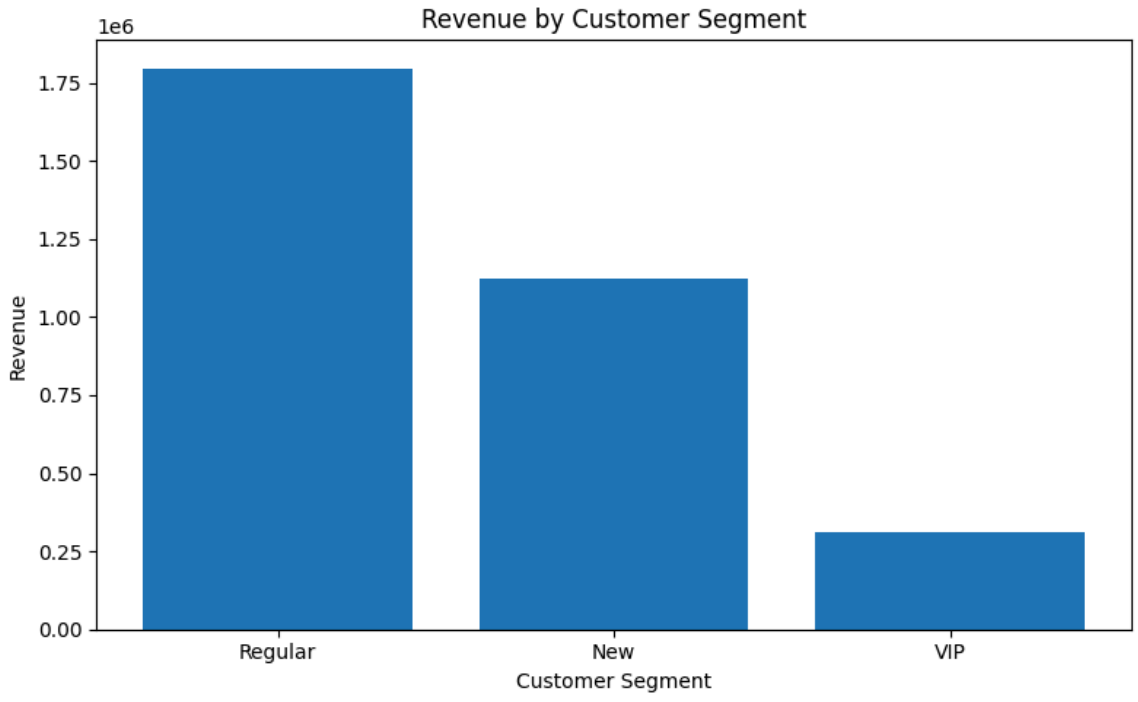
```
#Graph

plt.figure(figsize=(8, 5))

plt.bar(
    segment_analysis["CustomerSegment"],
    segment_analysis["Revenue"]
)

plt.title("Revenue by Customer Segment")
plt.xlabel("Customer Segment")
plt.ylabel("Revenue")

plt.tight_layout()
plt.show()
```



#Order status

For this analysis we do not filter to Completed + Paid, because we're trying to understand the overall order status.

```
status_analysis = (
    df["Status"]
    .value_counts()
    .reset_index()
)

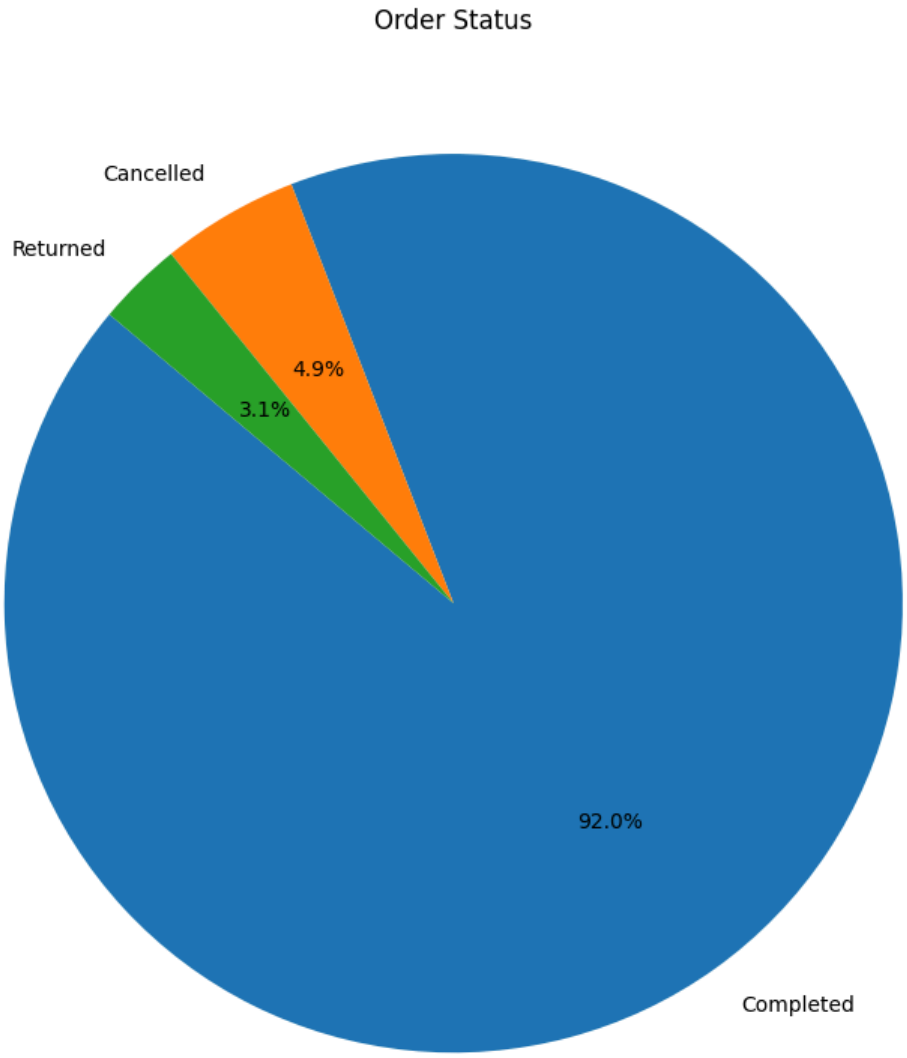
status_analysis.columns = [
    "Status",
    "Orders"
]


#Calculate the percentage
status_analysis["Percentage"] = (
    status_analysis["Orders"] /
    status_analysis["Orders"].sum()
) * 100

print(status_analysis)
```

```
      Status  Orders  Percentage
0  Completed   45968   91.991195
1  Cancelled    2471    4.944967
2   Returned    1531    3.063838
```

```
plt.figure(figsize=(8, 8))  # A square aspect ratio works best for pie charts
plt.pie(
    status_analysis["Orders"],
    labels=status_analysis["Status"],
    autopct="%1.1f%%",         # Displays the percentage on each slice
    startangle=140             # Rotates the start of the pie chart for
         better layout
)
plt.title("Order Status")
plt.tight_layout()
plt.show()
```



#Payment Status

Same principle: use the full dataset, not the recognised-revenue dataset.

```
payment_status = (
    df["PaymentStatus"]
    .value_counts()
    .reset_index()
)

payment_status.columns = [
    "PaymentStatus",
    "Orders"
]

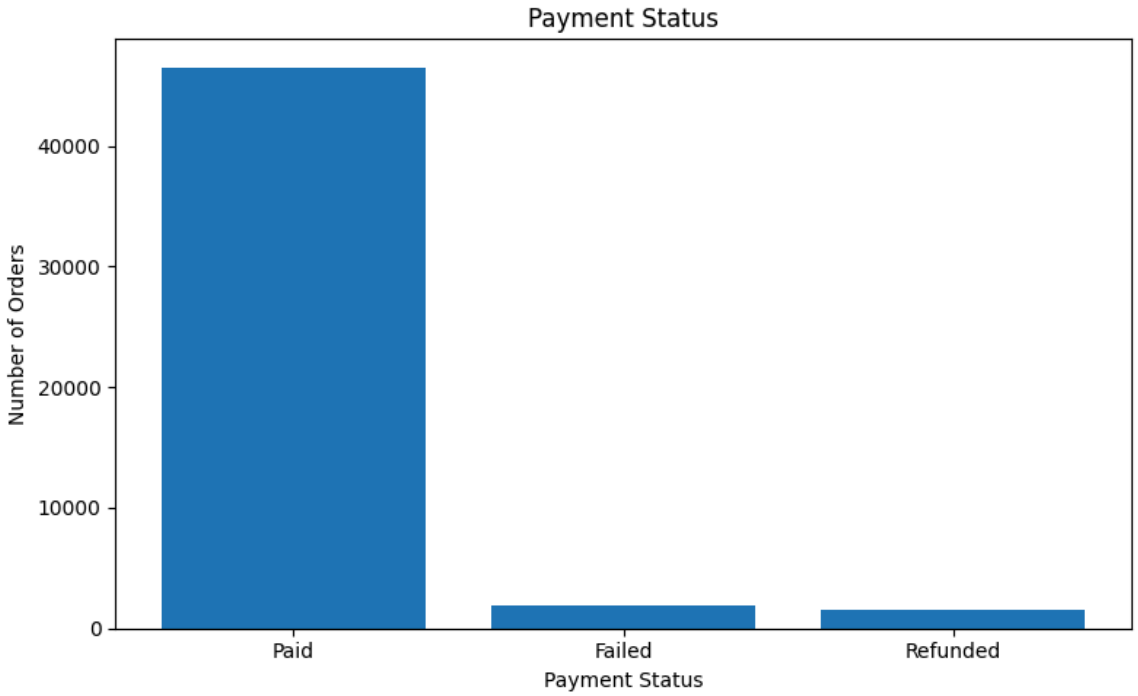
payment_status["Percentage"] = (
    payment_status["Orders"] /
    payment_status["Orders"].sum()
) * 100
```

```
#Graph
plt.figure(figsize=(8, 5))

plt.bar(
    payment_status["PaymentStatus"],
    payment_status["Orders"]
)

plt.title("Payment Status")
plt.xlabel("Payment Status")
plt.ylabel("Number of Orders")

plt.tight_layout()
plt.show()
```



#Payment Failure by Payment Method

This is slightly different.

We want:

Failed payments ÷ total payments for each payment method

```
payment_method_analysis = (
    df
    .groupby("PaymentMethod")
    .agg(
        Total_Payments=("PaymentStatus", "count"),
        Failed_Payments=(
            "PaymentStatus",
            lambda x: (x == "Failed").sum()
        )
    )
    .reset_index()
)
```

```
#Then

payment_method_analysis["Failure_Rate"] = (
    payment_method_analysis["Failed_Payments"] /
    payment_method_analysis["Total_Payments"]
) * 100

#Sort

payment_method_analysis = payment_method_analysis.sort_values(
    "Failure_Rate",
    ascending=False
)

print(payment_method_analysis)
```

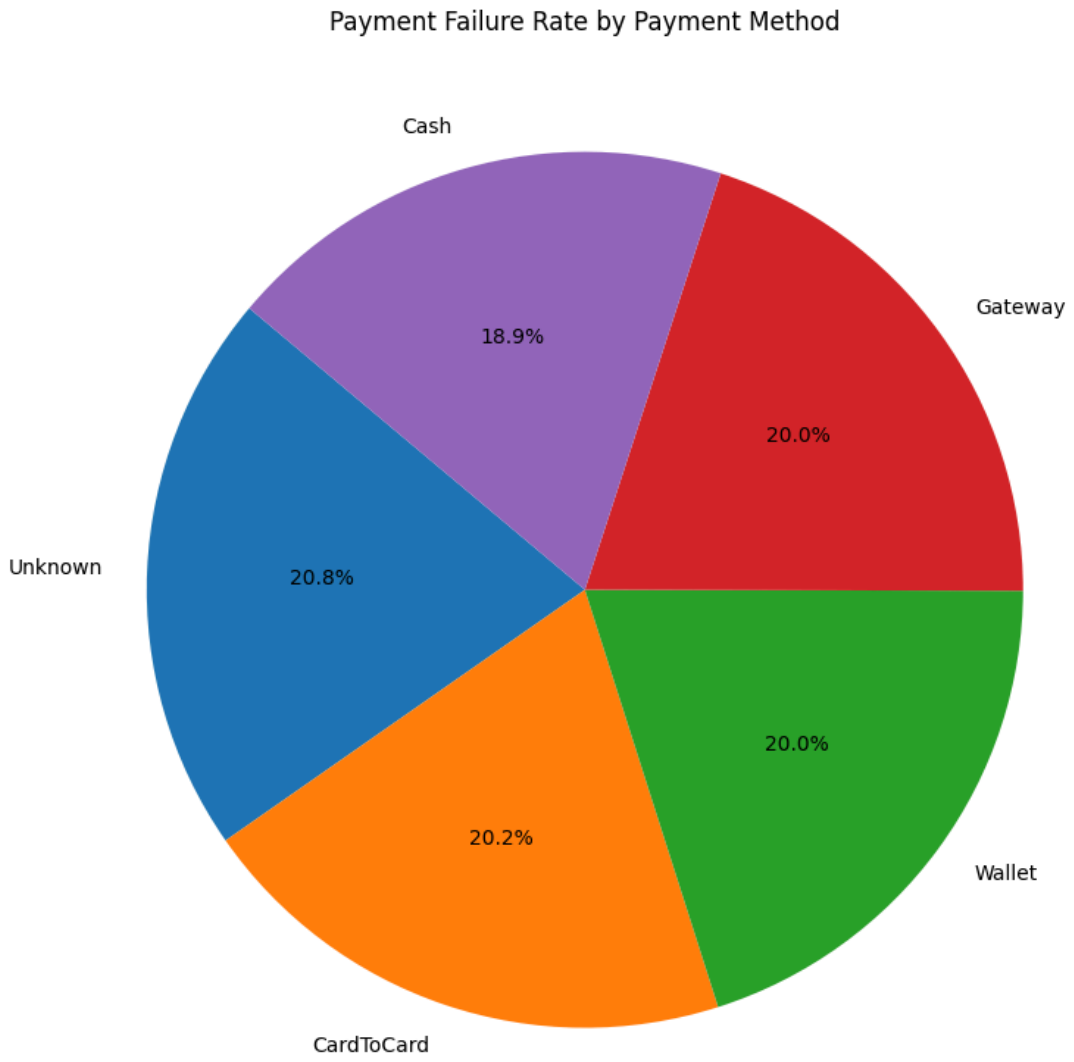
	PaymentMethod	Total_Payments	Failed_Payments	Failure_Rate
3	Unknown	449	18	4.008909
0	CardToCard	11938	465	3.895125
4	Wallet	8779	339	3.861488
2	Gateway	23807	918	3.856009
1	Cash	4997	182	3.642185

```
#Graph

plt.figure(figsize=(8, 8))  # Changed to a square ratio to keep the pie
         circular

plt.pie(
    payment_method_analysis["Failure_Rate"],
    labels=payment_method_analysis["PaymentMethod"],
    autopct="%1.1f%%",         # Displays the percentage share on each slice
    startangle=140             # Rotates the chart for a cleaner starting
         layout
)

plt.title("Payment Failure Rate by Payment Method")
plt.tight_layout()
plt.show()
```



#Discount Analysis

This is another analysis where we want to use the recognised-revenue dataset.

```
discount_analysis = (
    analysis_df
    .groupby("Discount")
    .agg(
        Revenue=("Revenue", "sum"),
        Orders=("OrderID", "nunique"),
        Units=("Quantity", "sum")
    )
    .reset_index()
)

#Calculate AOV
discount_analysis["AOV"] = (
    discount_analysis["Revenue"] /
    discount_analysis["Orders"]
)

#Calculate average units per order
```

```
discount_analysis["Avg_Units_Per_Order"] = (
    discount_analysis["Units"] /
    discount_analysis["Orders"]
)

#Sort by discount
discount_analysis = discount_analysis.sort_values(
    "Discount"
)

print(discount_analysis)
```

```
   Discount    Revenue  Orders    Units        AOV  Avg_Units_Per_Order
0      0.00  1122627.0   15066  28189.0  74.513939             1.871034
1      0.05   702331.0    9328  17693.0  75.292774             1.896762
2      0.10   647634.0    8460  15976.0  76.552482             1.888416
3      0.15   391839.0    5128   9644.0  76.411661             1.880655
4      0.20   234543.0    3062   5724.0  76.597975             1.869366
5      0.30   134357.0    1676   3148.0  80.165274             1.878282
```

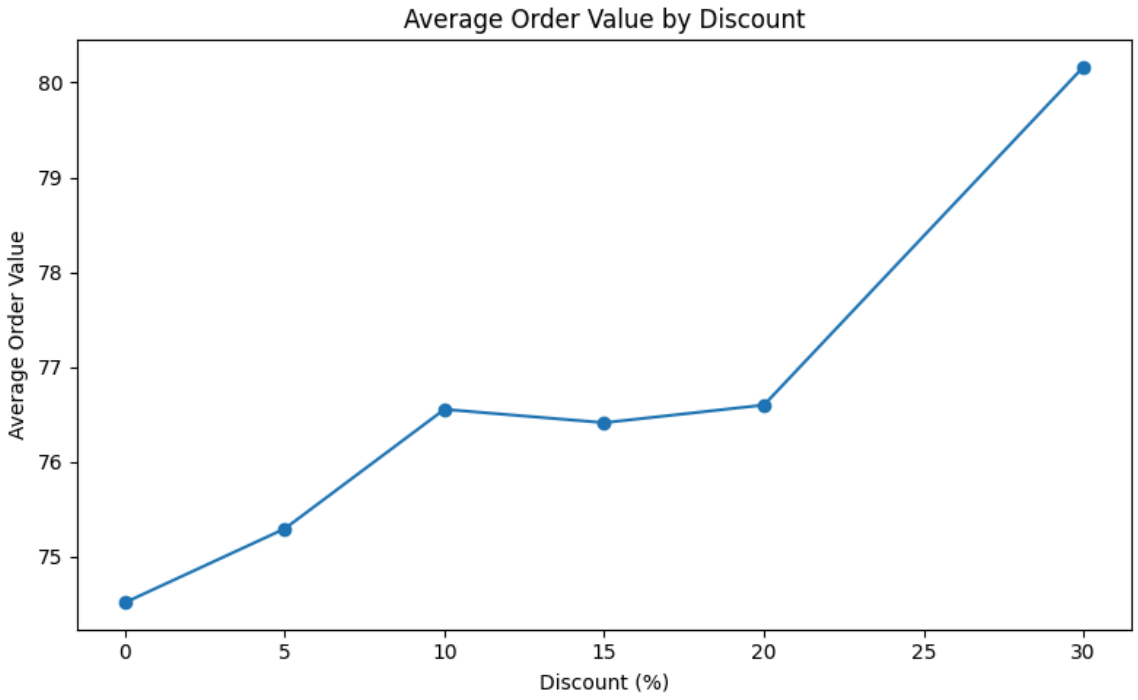
```
#Graph

plt.figure(figsize=(8, 5))

plt.plot(
    discount_analysis["Discount"] * 100,
    discount_analysis["AOV"],
    marker="o"
)

plt.title("Average Order Value by Discount")
plt.xlabel("Discount (%)")
plt.ylabel("Average Order Value")

plt.tight_layout()
plt.show()
```



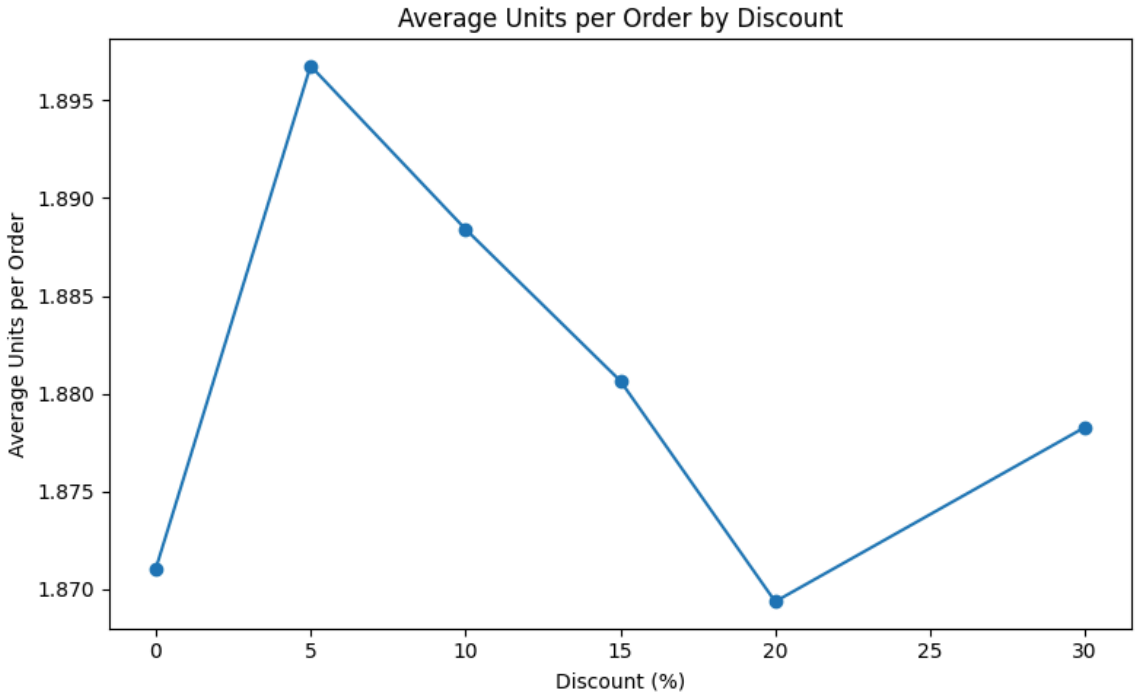
This supports the observation that higher discount levels generally correspond with lower AOV in this dataset, but we should not claim that discounts caused the lower AOV.

```
plt.figure(figsize=(8, 5))

plt.plot(
    discount_analysis["Discount"] * 100,
    discount_analysis["Avg_Units_Per_Order"],
    marker="o"
)

plt.title("Average Units per Order by Discount")
plt.xlabel("Discount (%)")
plt.ylabel("Average Units per Order")

plt.tight_layout()
plt.show()
```



This is useful because the business question asks whether:

Bigger discounts lead to bigger orders, or simply lower revenue?

#MATPLOTLIB DASHBOARD

```
#plt.subplots()

fig = plt.figure(figsize=(18, 14))

gs = fig.add_gridspec(
    3,
    3
)

#We can display them as cards.
fig = plt.figure(figsize=(18, 12))
```

```
gs = fig.add_gridspec(
    3,
    3,
    height_ratios=[1, 2, 2]
)

#Then
ax1 = fig.add_subplot(gs[0, 0])
ax2 = fig.add_subplot(gs[0, 1])
ax3 = fig.add_subplot(gs[0, 2])

#Make the first KPI card
ax1.text(
    0.5,
    0.5,
    f"R {total_revenue:,.2f}",
    ha="center",
    va="center",
    fontsize=22,
    fontweight="bold"
)

ax1.set_title(
    "Total Revenue",
    fontsize=14
)

ax1.axis("off")

#2nd KPI
ax2.text(
    0.5,
    0.5,
    f"{total_orders:,}",
    ha="center",
    va="center",
    fontsize=22,
    fontweight="bold"
)

ax2.set_title(
    "Total Orders",
    fontsize=14
)

ax2.axis("off")

#AOV KPI
ax3.text(
    0.5,
    0.5,
    f"R {aov:,.2f}",
    ha="center",
    va="center",
    fontsize=22,
    fontweight="bold"
)
```

```
ax3.set_title(
    "Average Order Value",
    fontsize=14
)

ax3.axis("off")

#Monthly revenue chart
monthly_revenue = (
    analysis_df
    .groupby("Year_Month")
    .agg(
        Revenue=("Revenue", "sum"),
        Orders=("OrderID", "nunique"),
        Units=("Quantity", "sum")
    )
    .reset_index()
)
ax4 = fig.add_subplot(gs[1, :])

ax4.plot(
    monthly_revenue["Year_Month"],
    monthly_revenue["Revenue"],
    marker="o"
)

ax4.set_title(
    "Monthly Revenue Trend",
    fontsize=16,
    fontweight="bold"
)

ax4.set_xlabel("Month")
ax4.set_ylabel("Revenue")

ax4.tick_params(axis="x", rotation=45)


#Category chart
ax5 = fig.add_subplot(gs[2, 0])

ax5.bar(
    category_analysis["Category"],
    category_analysis["Revenue"]
)

ax5.set_title(
    "Revenue by Category",
    fontweight="bold"
)

ax5.set_xlabel("Category")
ax5.set_ylabel("Revenue")

ax5.tick_params(
    axis="x",
    rotation=45
)
```

```
#Customer segement chart
ax6 = fig.add_subplot(gs[2, 1])

ax6.bar(
    segment_analysis["CustomerSegment"],
    segment_analysis["Revenue"]
)

ax6.set_title(
    "Revenue by Customer Segment",
    fontweight="bold"
)

ax6.set_xlabel("Customer Segment")
ax6.set_ylabel("Revenue")

#Order status chart
ax7 = fig.add_subplot(gs[2, 2])

ax7.bar(
    status_analysis["Status"],
    status_analysis["Orders"]
)

ax7.set_title(
    "Order Status",
    fontweight="bold"
)

ax7.set_xlabel("Status")
ax7.set_ylabel("Orders")

#fit to order
plt.tight_layout()
plt.show()
```

```
<Figure size 1800x1400 with 0 Axes>
```

Total Revenue	Total Orders	Average Order Value
R 3,233,331.00	**42,720**	**R 75.69**

